

ОТЧЁТ О ПРОХОЖДЕНИИ ПРОИЗВОДСТВЕННОЙ ПРАКТИКИ

Студенты: Рязанцев Данил Николаевич и Рыбалко Лев Павлович

Группа: Кейс #3 – Kanban-14

Место прохождения практики: VK (ООО «ВКонтакте»)

Руководитель практики от организации: Кулис Максим

Период практики: 06.07.2026 – 23.07.2026

Москва, 2026

ОГЛАВЛЕНИЕ

1. Введение (предметная область)	2
2. Анализ требований к системе	2
3. Этапы выполнения работы	4
4. Структура базы данных	7
5. API и схемы данных	11
6. Выводы	15
7. Источники	16

1. ВВЕДЕНИЕ (ПРЕДМЕТНАЯ ОБЛАСТЬ)

1.1. Описание предметной области

Предметной областью является управление проектами и задачами с использованием методологии Kanban. Kanban — это метод управления рабочим процессом, который визуализирует задачи на доске, разделённой на колонки, соответствующие этапам выполнения.

1.2. Основные сущности

- **Доска** — рабочее пространство для управления задачами.
- **Колонка** — этап выполнения задач (например, "Новые", "В работе", "Готово").
- **Карточка** — отдельная задача с описанием, исполнителем и дедлайном.
- **Комментарий** — обсуждение задачи.
- **Пользователь** — участник системы.

2. АНАЛИЗ ТРЕБОВАНИЙ К СИСТЕМЕ

2.1. Функциональные требования

ID	Требование	Приоритет
FR-01	Регистрация и авторизация пользователей	Высокий
FR-02	Создание, редактирование, удаление досок	Высокий
FR-03	Создание, редактирование, удаление колонок	Высокий
FR-04	Создание, редактирование, удаление карточек	Высокий
FR-05	Перемещение карточек между колонками (Drag-and-Drop)	Высокий
FR-06	Назначение исполнителя и дедлайна	Средний

FR-07	Добавление комментариев к карточкам	Средний
FR-08	Система управления доступом (RBAC + ABAC)	Средний
FR-09	Тёмная тема оформления	Низкий

2.2. Нефункциональные требования

Требование	Описание
Производительность	Время ответа API не более 500 мс
Надёжность	Защита от потери данных
Безопасность	Хэширование паролей, JWT-авторизация
Масштабируемость	Поддержка множества пользователей и досок
Удобство использования	Интуитивно понятный интерфейс

2.3. Технологический стек

Компонент	Технология	Обоснование
Фронтенд	React 18	Популярность, компонентный подход, большая экосистема
Маршрутизация	React Router v6	Стандарт для React-приложений
Drag-and-Drop	@dnd-kit	Простота интеграции, хорошая документация

Бэкенд	Python + FastAPI	Высокая производительность, автоматическая документация
ORM	SQLAlchemy	Мощный инструмент для работы с БД
База данных	PostgreSQL	Надёжность, поддержка сложных запросов
Аутентификация	JWT + bcrypt	Безопасность, стандарт индустрии

3. ЭТАПЫ ВЫПОЛНЕНИЯ РАБОТЫ

3.1. План-график работ

Этап	Период	Описание	Результат
1	08.07 – 09.07	Проектирование архитектуры	Схема БД, структура API
2	09.07 – 15.07	Разработка бэкенда	Реализованы все API-ручки
3	09.07 – 15.07	Разработка фронтенда	Реализованы все страницы
4	15.07 – 16.07	Интеграция и тестирование	Работающее приложение
5	15.07 – 16.07	Документирование	Отчёт, README, демо

3.2. Выполненные работы

Этап 1. Проектирование архитектуры

- Разработана схема базы данных в PostgreSQL.
- Определены все API-эндпоинты.
- Выбраны технологии для реализации.

Этап 2. Разработка бэкенда

- Реализованы модели данных на SQLAlchemy.

- Созданы Pydantic-схемы для валидации.
- Реализованы все API-ручки.
- Настроена JWT-аутентификация.
- Реализована система управления доступом (RBAC + ABAC).

Этап 3. Разработка фронтеда

- Созданы страницы: логин, список досок, доска.
- Реализованы компоненты: колонка, карточка, модальное окно.
- Интегрирован Drag-and-Drop (@dnd-kit).
- Реализована тёмная тема.

Этап 4. Интеграция и тестирование

- Настроено взаимодействие фронта и бэка.
- Проведено тестирование всех сценариев.
- Исправлены выявленные ошибки.

3.3. Система управления доступом (RBAC + ABAC)

В проекте реализована гибридная система управления доступом:

RBAC (Role-Based Access Control):

Роль	Описание
admin	Полный доступ к доске (удаление, редактирование)
member	Создание и редактирование карточек
viewer	Только просмотр

ABAC (Attribute-Based Access Control):

- Исполнитель карточки может редактировать свою карточку, даже если не является admin или member.
- Проверка: `card.assigned_to == current_user.id`

Реализация в коде:

python

```
if board.owner_id != current_user.id:
```

```

participant = db.execute(...).first()
if not participant or participant.role not in ["admin", "member"]:
    if card.assigned_to != current_user.id:
        raise HTTPException(status_code=403)

```

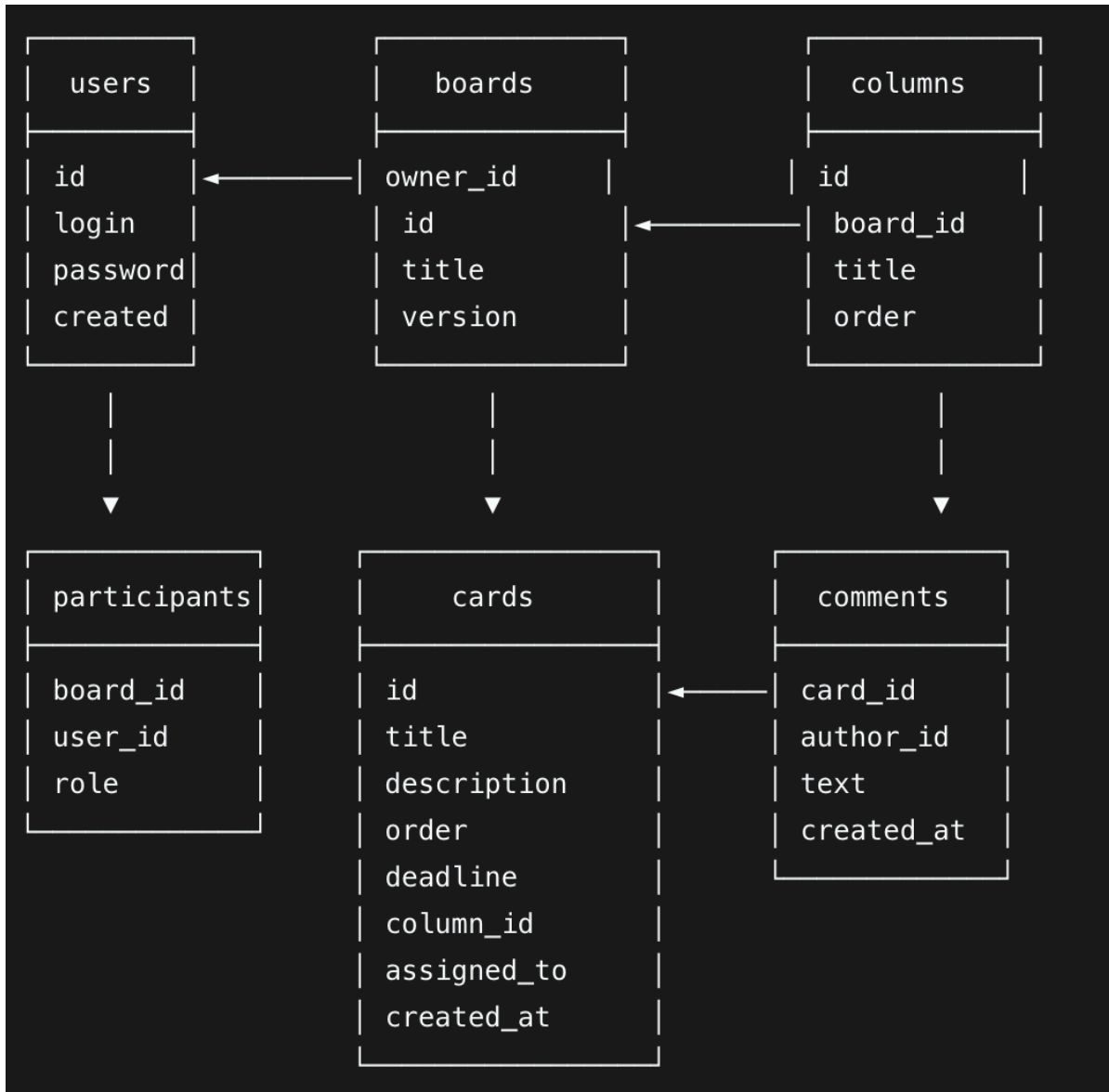
Все проверки выполняются на бэкенде, что соответствует принципу безопасности.

3.4. Возникшие проблемы и их решение

Проблема	Решение
Ошибка CORS	Настроен CORS-middleware в FastAPI
Ошибка 422 при создании карточки	Добавлено поле <code>column_id</code> в <code>CardCreate</code>
Не работал Drag-and-Drop	Исправлена передача данных в <code>useSortable</code>
Двойное создание досок	Добавлена защита от дубликатов

4. СТРУКТУРА БАЗЫ ДАННЫХ

4.1. ER-диаграмма



4.2. Описание таблиц

Таблица users

Поле	Тип	Описание
id	UUID	Первичный ключ
login	VARCHAR(255)	Логин пользователя (уникальный)

password VARCHAR(255) Хэш пароля (bcrypt)

created_at TIMESTAMP Дата регистрации

Таблица boards

Поле	Тип	Описание
id	UUID	Первичный ключ
title	VARCHAR(255)	Название доски
owner_id	UUID	Владелец (FK → users)
version	INTEGER	Версия (защита от коллизий)
created_at	TIMESTAMP	Дата создания

Таблица board_participants

Поле	Тип	Описание
board_id	UUID	ID доски (FK → boards)
user_id	UUID	ID пользователя (FK → users)
role	VARCHAR(50)	Роль: admin, member, viewer

Таблица columns

Поле	Тип	Описание
id	UUID	Первичный ключ
title	VARCHAR(255)	Название колонки
order	INTEGER	Порядок отображения

board_id	UUID	ID доски (FK → boards)
----------	------	------------------------

Таблица cards

Поле	Тип	Описание
id	UUID	Первичный ключ
title	VARCHAR(255)	Название карточки
description	TEXT	Описание
order	INTEGER	Порядок в колонке
deadline	TIMESTAMP	Дедлайн
column_id	UUID	ID колонки (FK → columns)
assigned_to	UUID	Исполнитель (FK → users)
created_at	TIMESTAMP	Дата создания

Таблица comments

Поле	Тип	Описание
id	UUID	Первичный ключ
text	TEXT	Текст комментария
card_id	UUID	ID карточки (FK → cards)
author_id	UUID	Автор (FK → users)
created_at	TIMESTAMP	Дата создания

4.3. Индексы

sql

```
CREATE INDEX idx_cards_column_id ON cards(column_id);  
CREATE INDEX idx_cards_assigned_to ON cards(assigned_to);  
CREATE INDEX idx_comments_card_id ON comments(card_id);  
CREATE INDEX idx_board_participants_user_id ON board_participants(user_id);  
CREATE INDEX idx_boards_owner_id ON boards(owner_id);
```

5. API И СХЕМЫ ДАННЫХ

5.1. Схемы данных (Pydantic)

```
1 from pydantic import BaseModel
2 from datetime import datetime
3 from typing import Optional
4
5 # AUTH
6 class UserCreate(BaseModel):
7     login: str
8     password: str
9
10 class UserLogin(BaseModel):
11     login: str
12     password: str
13
14 class Token(BaseModel):
15     access_token: str
16     token_type: str = "bearer"
17
18 # BOARDS
19 class BoardCreate(BaseModel):
20     title: str
21
22 class BoardUpdate(BaseModel):
23     title: str
24     version: int
25
26 class BoardResponse(BaseModel):
27     id: str
28     title: str
29     owner_id: str
30     version: int
31     created_at: datetime
32
33 # COLUMNS
34 class ColumnCreate(BaseModel):
35     title: str
36
37 class ColumnUpdate(BaseModel):
38     title: str
39
40 class ColumnResponse(BaseModel):
41     id: str
42     title: str
43     order: int
44     board_id: str
45
46 # CARDS
47 class CardCreate(BaseModel):
48     title: str
49     description: Optional[str] = None
50     assigned_to: Optional[str] = None
51     deadline: Optional[datetime] = None
52
53 class CardUpdate(BaseModel):
54     title: Optional[str] = None
55     description: Optional[str] = None
56     assigned_to: Optional[str] = None
57     deadline: Optional[datetime] = None
58
59 class CardMove(BaseModel):
60     target_column_id: str
61     new_order: int
62
63 # COMMENTS
64 class CommentCreate(BaseModel):
65     text: str
```

5.2. API-эндпоинты

Аутентификация

Метод	Путь	Описание
POST	/auth/register	Регистрация пользователя
POST	/auth/login	Вход (выдача JWT)

Доски

Метод	Путь	Описание
GET	/boards	Получить все доски пользователя
POST	/boards	Создать доску
PATCH	/boards/{id}	Обновить доску
DELETE	/boards/{id}	Удалить доску

Колонки

Метод	Путь	Описание
GET	/columns/board/{board_id}	Получить колонки доски
POST	/columns?board_id={board_id}	Создать колонку
PATCH	/columns/{id}	Обновить колонку
DELETE	/columns/{id}	Удалить колонку

Карточки

Метод	Путь	Описание
GET	/cards/column/{column_id}	Получить карточки колонки

POST	/cards	Создать карточку
PATCH	/cards/{id}	Обновить карточку
DELETE	/cards/{id}	Удалить карточку
E		
PATCH	/cards/{id}/move	Переместить карточку

Комментарии

Метод	Путь	Описание
GET	/comments/card/{card_id}	Получить комментарии
POST	/comments?card_id={card_id}	Добавить комментарий
DELETE	/comments/{id}	Удалить комментарий
E		

5.3. Пример API-запроса на фронтенд

```
// ===== ОСНОВНОЙ API =====
export const api = {
  // Auth
  register: async (login, password) => {
    console.log(' api.register вызван с:', login, password);
    if (USE MOCK) {
      return mockAuth.register(login, password);
    }
    return fetch(`${API_URL}/auth/register`, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ login, password })
    }).then(r => r.json());
  },

  login: async (login, password) => {
    console.log(' api.login вызван с:', login, password);
    if (USE MOCK) {
      return mockAuth.login(login, password);
    }
    return fetch(`${API_URL}/auth/login`, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ login, password })
    }).then(r => r.json());
  },

  // Boards
  getBoards: async () => {
    if (USE MOCK) return mockCrud.getBoards();
    return authFetch('/boards');
  },

  createBoard: async (title) => {
    if (USE MOCK) return mockCrud.createBoard(title);
    return authFetch('/boards', { method: 'POST', body: { title } });
  },

  updateBoard: async (id, title, version) => {
    if (USE MOCK) return mockCrud.updateBoard(id, title, version);
    return authFetch(`/boards/${id}`, { method: 'PATCH', body: { title, version } });
  },

  deleteBoard: async (id) => {
    if (USE MOCK) return mockCrud.deleteBoard(id);
    return authFetch(`/boards/${id}`, { method: 'DELETE' });
  },
}
```

5.3. API-запросы бекенд

auth		^
POST	/auth/register Register	✓
POST	/auth/login Login	✓
boards		^
GET	/boards/ Get Boards	🔒 ✓
POST	/boards/ Create Board	🔒 ✓
PATCH	/boards/{board_id} Update Board	🔒 ✓
DELETE	/boards/{board_id} Delete Board	🔒 ✓
columns		^
GET	/columns/board/{board_id} Get Columns	📱 🔒 ✓
POST	/columns/ Create Column	🔒 ✓
PATCH	/columns/{column_id} Update Column	🔒 ✓
DELETE	/columns/{column_id} Delete Column	🔒 ✓
cards		^
GET	/cards/column/{column_id} Get Cards	🔒 ✓

6. ВЫВОДЫ

6.1. Итоги практики

За время производственной практики была разработана Kanban-доска со следующим функционалом:

- Регистрация и авторизация пользователей с JWT.
- Управление досками, колонками и карточками.
- Drag-and-Drop для перемещения карточек.
- Назначение исполнителей и дедлайнов.
- Комментарии к карточкам.
- Гибридная система управления доступом (RBAC + ABAC).
- Тёмная тема оформления.

6.2. Навыки, полученные на практике

Навык	Уровень
Разработка на React	Продвинутый

Разработка на FastAPI	Продвинутый
Проектирование БД	Средний
Работа с JWT	Средний
Работа с Git	Базовый
Документирование API	Средний

6.3. Результаты

- Разработано веб-приложение для управления задачами.
- Получен опыт работы с современными технологиями.
- Закреплены теоретические знания на практике.

7. ИСТОЧНИКИ

1. **React Documentation.** – URL: <https://react.dev/> (дата обращения: 14.07.2026).
2. **FastAPI Documentation.** – URL: <https://fastapi.tiangolo.com/> (дата обращения: 14.07.2026).
3. **SQLAlchemy Documentation.** – URL: <https://www.sqlalchemy.org/> (дата обращения: 14.07.2026).
4. **PostgreSQL Documentation.** – URL: <https://www.postgresql.org/docs/> (дата обращения: 14.07.2026).
5. **dnd-kit Documentation.** – URL: <https://dndkit.com/> (дата обращения: 14.07.2026).
6. **React Router Documentation.** – URL: <https://reactrouter.com/> (дата обращения: 14.07.2026).
7. **JWT Introduction.** – URL: <https://jwt.io/introduction> (дата обращения: 14.07.2026).
8. **RBAC and ABAC Overview.** – URL: <https://habr.com/ru/companies/custis/articles/248649/> (дата обращения: 14.07.2026).